

Server module, as follows:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>de.codecentric</groupId>
  <artifactId>spring-boot-admin-server-ui-login</
artifactId>
  <version>1.5.1</version>
</dependency>
```

Add the properties given below to the `application.properties` file.

```
security.user.name=admin //user name to authenticate
security.user.password=admin123 //Password to authenticate
```

As we have added security to the Admin Server, admin clients should be able to connect to the server by

authenticating. Hence, add the properties given below to the admin client's `application.properties` files.

```
spring.boot.admin.username=admin
spring.boot.admin.password=admin123
```

There are additional UI features like Hystrix and Turbine UI, which we can enable in the dashboard. You can find more details at http://codecentric.github.io/spring-boot-admin/1.5.1/#_ui_modules. The sample code created for this demonstration is available on https://github.com/2013techsmarts/SpringBoot_Admin_Demo. 

By: Siva Prasad Rao Janapati

The author is a software engineer with hands-on experience in Java, JEE, Spring, Oracle Commerce, MOZU Commerce, Apache Solr and other open source/enterprise technologies. You can reach him at his blog <http://smarttechie.org>.

Continued from page 60....

Selenium locators

Locator is a command that instructs the Selenium IDE which GUI element it needs to work on. Elements are located in Selenium WebDriver with the help of `findElement()` and `findElements()` methods provided by the WebDriver and WebElement class. The `findElement()` method returns a WebElement object based on a specified search criteria or ends up throwing an exception. The `findElements()` method returns a list of WebElements matching the search criteria. If these are not found, it returns an empty list.

The different types of locators are:

1. ID
2. Name
3. Link Text
4. CSS Selector
5. DOM
6. XPath

To locate by ID, type:

```
driver.findElement(By.id(<element ID>));
```

To locate by name, type:

```
driver.findElement(By.name(<element name>));
```

To locate by Link Text, type:

```
driver.findElement(By.linkText(<linktext>));
```

To locate by CSS Selector, type:

```
driver.findElement(By.cssSelector(<css selector>));
```

To locate by XPath, type:

```
driver.findElement(By.xpath(<xpath>));
```

Limitations of Selenium

Selenium does have some limitations which one needs to be aware of. First and foremost, image based testing is not clear-cut compared to some other commercial tools in the market, while the fact that it is open source also means that there is no guaranteed timely support. Another limitation of Selenium is that it supports Web applications; therefore, it is not possible to automate the testing of non-browser based applications.

Selenium is a power testing framework to conduct functional and regression testing. It is open source software and supports various programming environments, OSs and popular browsers. Selenium WebDriver is used to conduct batch testing, cross-platform browser testing, data driven testing, etc. It is also very cost-effective when automating Web applications; and for the technically inclined, it provides the power and flexibility to extend its capability many times over, making it a very credible alternative to other test automation tools in the market. 

By: Neetesh Mehrotra

The author works in TCS as a systems engineer. His areas of interest are Java development and automation testing. You can contact him at mehrotra.neetesh@gmail.com.